

1. CI/CD Architecture Overview

The CI/CD architecture created for the ACEest Fitness & Gym application ensures that the development process is completely automated from the moment code is committed to the repository until the final deployment stage. The pipeline incorporates core DevOps tools including GitHub for version control, Jenkins for continuous integration, Pytest for automated testing, SonarQube for code quality analysis, Docker for containerization, and Kubernetes (Minikube) for deployment and orchestration. The architecture ensures unified automation, consistent testing, reproducible builds, and reliable deployment strategies.

The process begins when developers push updated application versions into GitHub. A multibranch Jenkins pipeline continuously monitors the repository. Once Jenkins detects changes, it triggers the CI pipeline, which installs dependencies, configures Python environments, runs test cases, performs code quality checks, builds artifacts, and prepares the application for containerization. Pytest ensures that every new feature or update behaves as expected, while SonarQube scans for vulnerabilities, code smells, and maintainability issues. Only if the quality checks pass does the pipeline proceed to the next stages.

The Dockerfile packages the app and all its dependencies into a standardized Docker image. These containerized images are tagged by version and stored on Docker Hub, ensuring traceability across releases. For production-like deployment, Minikube orchestrates the application using Kubernetes deployments, services, and rolling update strategies. This final stage ensures reliability, fault■tolerance, and scalability. The architecture follows industry■grade DevOps workflows supporting continuous integration, continuous deployment, and automated quality enforcement.

2. Challenges Faced and Mitigation Strategies

Several challenges were encountered throughout the implementation of the CI/CD pipeline, especially due to running the entire system on a Windows environment. One major challenge involved Docker Desktop, which repeatedly failed due to an outdated WSL2 kernel. Docker relied heavily on WSL2, therefore updating WSL via 'wsl --update', enabling virtualization in BIOS, and restarting the system resolved the issue completely.

Another major challenge involved Jenkins authentication with GitHub. Initially, the GitHub interface did not show the Personal Access Token section because it is located under Developer Settings rather than repository settings. Once identified, a PAT was generated and stored securely in Jenkins Credentials Manager. This token enabled Jenkins to clone repositories, scan branches, and trigger multibranch pipelines correctly.

Testing■related issues also appeared, especially the "ModuleNotFoundError: No module named 'app'" error during Pytest execution. This occurred because Python modules were not properly structured.

The issue was resolved by introducing a clear folder structure with a dedicated 'src' folder, creating a pytest.ini file, and defining pythonpath in it. After restructuring, tests executed successfully.

Jenkins also failed to detect Python installations using Pyenv. This issue was resolved by configuring Python explicitly in Jenkins Global Tool Configuration and calling the selected Python runtime inside the Jenkinsfile. SonarQube integration also failed initially due to unauthorized scanner access. Generating a Sonar token and storing it securely in Jenkins credentials enabled proper communication between Jenkins and SonarQube.

Containerization challenges arose because the application used Tkinter GUI, which is not ideal for container environments. The solution was to modify the app to run without GUI during Docker builds and tests. Additionally, Kubernetes deployment produced YAML formatting and indentation issues, which prevented pods from starting. After correcting the YAML, defining proper selectors, and enabling the correct imagePullPolicy, Kubernetes successfully deployed the application using rolling updates.

3. Key Automation Outcomes

The completed CI/CD pipeline delivered several strong automation outcomes that significantly improved the reliability, maintainability, and overall performance of the ACEest Fitness & Gym application development lifecycle. The most important achievement was establishing a fully automated CI pipeline that builds, tests, and analyzes code with every push. This reduced manual intervention and minimized human error.

Automated testing through Pytest ensured continuous validation of application functionality. Failures were detected early, reducing debugging time and preventing unstable versions from progressing further down the pipeline. Jenkins also published readable test reports, supporting better visibility for developers.

SonarQube integration introduced automated code quality enforcement. Vulnerabilities, bad practices, and duplicate code were identified and fixed early. Quality gates prevented poor-quality code from reaching production environments.

Docker containerization resulted in reproducible environments. Every application version was packaged with its dependencies, eliminating inconsistencies across different machines. Version-tagged images stored in Docker Hub allowed easy rollback and consistent deployment processes.

Kubernetes deployment added a final layer of reliability. Rolling updates allowed new versions to be deployed without downtime. Kubernetes replicas ensured continuous availability, while quick rollback minimized risk in case of failures. This created a production-grade release process suitable for scalable environments.

4. Conclusion

The DevOps pipeline established for ACEest Fitness & Gym successfully automated the entire application lifecycle using modern industry tools. Each component—from GitHub commits to Kubernetes deployment—worked together to deliver a secure, testable, maintainable, and scalable release process. The implementation strengthened development efficiency, ensured consistent code quality, and demonstrated real-world DevOps practices in action.